

Bienvenido al manual paso a paso de **RACK Designer Next**, tu herramienta definitiva para gestionar salas, racks, equipos de telecomunicaciones y conexiones. Este manual está diseñado asumiendo que **no tienes experiencia previa** con la aplicación, por lo que te guiaremos desde lo más básico hasta que seas capaz de mapear un centro de datos completo.

 Parte 1: Conceptos Básicos (Para Principiantes)

Antes de hacer clic en cualquier botón, es vital entender qué estamos construyendo.

¿Qué es un Datacenter o Sala de Equipos?

Es simplemente una habitación física donde se guardan las computadoras centrales (servidores), los aparatos que reparten internet (switches, routers) y los sistemas de energía (baterías, aires acondicionados). En nuestra aplicación, todo empieza por crear una **Sala**.

¿Qué es un "Rack" o "Gabinete"?

Imagina un estante metálico, como un librero muy alto y estandarizado. Su propósito es apilar equipos uno encima del otro para ahorrar espacio. En nuestra aplicación, los Racks van **adentro de las Salas**.

¿Qué es una Unidad "U"?

Como los estantes del "Rack" están vacíos, necesitamos medir la altura de los equipos para saber cuántos caben. Para ello usamos la medida **"U"**.

- Un Rack de tamaño completo suele tener **42U** de espacio de arriba a abajo.
- Un equipo pequeño ocupa **1U**. Un servidor más grande puede ocupar **2U** o **4U**.

[ITIP] **Vistas de un Rack (Frente y Atrás)** Los Racks tienen dos puertas. Por delante (Front) sueles ver los botones de encendido y discos duros. Por detrás (Rear) sueles ver los cables de poder y puertos de red. ¡La aplicación te permite girar el Rack para trabajar por ambas caras!

 Parte 2: Conociendo la Interfaz

Al abrir RACK Designer Next, verás tu área de trabajo dividida en **5 paneles principales**.

1. **Barra Superior (Header):** Aquí administras tu "Rol" (para iniciar sesión como Admin y tener permisos para editar), el botón para Exportar tu plano a Excel, y el botón para Guardar el archivo en tu computadora.
2. **Catálogo (Izquierda):** Es tu "tienda" de equipos. Hay categorías para Servidores, Switches, Energía (PDU), y Equipos de Piso. Desde aquí **arrastrarás** los equipos al centro.
3. **Área de Trabajo (Centro):** El lienzo principal. Aquí es donde los Racks cobran vida. En la parte superior de esta área (Cabecera), encontrarás los menús desplegables para seleccionar y crear Salas ("Data Center") y Racks, junto con las pestañas unificadas para cambiar entre **Vista Física** y **Topología**.
4. **Inspector y Árbol (Derecha):** Cuando seleccionas un Rack o un servidor, en este panel verás su nombre, dirección IP, número de serie y otros detalles. También puedes cambiarlos.
5. **Inventario (Abajo):** Una gran tabla tipo Excel que lista todos los equipos que has colocado en tu sala, además de listar los cables y puertos conectados.

 Parte 3: Flujo de Trabajo y Operaciones Comunes

La aplicación fue diseñada para ser tan fácil como un juego de "arrastrar y soltar".

Cómo moverse y seleccionar

- **Clic Izquierdo:** Selecciona un objeto (Rack, servidor, etc.). Al seleccionarlo, se iluminará y sus datos aparecerán en el panel de la derecha.
- **Rueda del Ratón:** Haz scroll para hacer acercar (Zoom In) o alejar (Zoom Out).
- **Arrastrar (Drag & Drop):** Haz clic sostenido en un equipo del catálogo izquierdo, muévelo hasta un hueco vacío en tu Rack, y suelta el botón.

Guardar y Cargar

La aplicación guarda **automáticamente** los cambios en la memoria temporal de tu navegador cada vez que haces un movimiento. Sin embargo, para no perder tu trabajo, debes descargar el archivo:

1. Ve a la **Barra Superior**.
2. Haz clic en **Opciones del Proyecto -> Exportar (Save As)**.
3. Se descargará un archivo con extensión .rack. ¡Ese es tu proyecto!
4. Para abrirlo mañana, usa **Importar** y selecciona ese mismo archivo.

 Parte 4: Tutoriales Prácticos Paso a Paso

¡Hora de la práctica! Vamos a crear tres salas distintas desde cero.

Escenario 1: Datacenter Híbrido (Racks y Piso)

Vamos a crear una sala normal, que contiene 2 Racks con servidores, y adicionalmente un equipo pesado (Aire Acondicionado) que va en el piso, fuera de los racks.

Paso 1: Crear la Sala y los Racks

1. En la **Barra Superior (Cabecera)**, verás un menú desplegable que dice "Data Center" y a su lado un botón "+".
2. Haz clic en el menú o selecciona la sala existente. En el panel Derecho (Inspector), cámbiale el nombre a Mi Datacenter Híbrido.
3. En la misma barra superior, junto al menú desplegable de "Racks", haz clic en el botón "+" para agregar un Rack. Aparecerá un cajón negro en el lienzo central.
4. En el panel Derecho, ponle nombre a ese rack: RACK-01.
5. Repite el paso 3 y 4 para crear otro rack llamado RACK-02.

Paso 2: Llenar el RACK-01

1. Ve al panel Izquierdo (Catálogo) y haz clic en la categoría **Servers** para expandirla.
2. Haz clic sostenido sobre un "Dell Server (2U)" y arrástralo hacia el RACK-01 en el lienzo central. Suéltalo en un hueco vacío.
3. Expande la categoría **Network**. Arrastra un "Switch 48P (1U)" y suéltalo justo arriba del servidor.

Paso 3: Añadir el equipo de piso

1. En el Catálogo, busca la categoría **Floor / Piso**.
2. Arrastra una "Unidad InRow (Aire Acondicionado)". Notarás que el sistema *no te deja* meterlo dentro del Rack.
3. **¿Cómo lo coloco en el piso?** Muy simple: En la parte inferior del Rack, verás una zona punteada llamada **"Floor Equip (Exterior)"**. Suelta el equipo ahí.
4. Haz clic en el aire acondicionado recién colocado y, en el panel Derecho, llámalo Aire-Principal. ¡Felicidades, completaste el escenario 1!

Escenario 2: Sala de Telecomunicaciones (Solo Racks)

Este cuarto más pequeño no tiene equipos en el piso, solo equipos de red y parcheo.

Paso 1: Crear una Sala nueva

1. Ve a la **Barra Superior (Cabecera)** y haz clic en el botón "+" que está al lado del selector de Salas ("Data Center").
2. En el panel Derecho, llámala Cuarto de Telecomunicaciones. Notarás que el lienzo central se vacía (porque entraste a tu nueva sala vacía).

Paso 2: Montar el Gabinete de Red

1. En la Barra Superior, haz clic en el botón "+" junto al selector de Racks y llámalo RACK-TELCO en el panel derecho.
2. Ve al Catálogo -> Categoría **Network**.
3. Arrastra un "Patch Panel (1U)" a la parte más alta del rack.
4. Arrastra un "Router Core (4U)" debajo del patch panel.
5. Ve al panel Derecho y en las propiedades del Router escribe en la IP: 192.168.1.1.

[INOTE] En este escenario no usamos equipos de piso. Tu vista principal (Topológica o Frontal) mostrará el cuarto ordenado y exclusivamente dedicado a equipos montados.

Escenario 3: Sala de Fuerza / Energía (Solo Equipos de Piso)

En el sótano del edificio, tenemos una sala de baterías gigantes y generadores que no caben en ningún rack.

Paso 1: Crear la Sala de Energía

- 1. En la Barra Superior, haz clic en el botón "+" junto al selector de Salas.
- 2. En el panel Derecho, nómbrala Cuarto de Baterías.

Paso 2: Llenar el cuarto sin usar Racks Dado que esta sala no necesita racks metálicos altos, insertaremos los equipos de manera independiente.

- 1. **NO** hagas clic en el botón "-" de Racks.
- 2. Ve directamente al Catálogo -> Categoría **Floor / Piso**.
- 3. Arrastra un "Standalone UPS" hacia el lienzo vacío central.
- 4. Como no hay un Rack, la aplicación creará automáticamente una zona invisible (un chasis contenedor de piso) para alojarlo.
- 5. Arrastra un "Desk / Escritorio" al lienzo.
- 6. En el panel Derecho, renombra los equipos a Batería-Principal y Mesa Operador.

¡Has completado tu capacitación básica! Ahora sabes cómo navegar por la aplicación, qué significan los conceptos, cómo arrastrar componentes y cómo estructurar cuartos enteros según tus necesidades del mundo real.

Si te equivocas, recuerda usar la opción de **Deshacer (Undo)** o simplemente seleccionar el equipo mal colocado y presionar el botón **Eliminar (Trash/Basurero)** en el Inspector (Panel Derecho).

 Parte 4: Visualización de Conexiones Físicas

¡Ahora también puedes visualizar cómo están conectados los cables en la vida real!

- 1. Crea una conexión entre dos equipos en la pestaña de Inventario ("+ Conexión").
- 2. Ve a la **Vista Física** (botón superior central).

- 3. Haz clic en el interruptor deslizable **"Cables"** de la barra de herramientas.
- 4. Verás que se dibujan automáticamente los cables desde los puertos, organizados ortogonalmente y agrupándose en los bordes del rack.
- 5. Usa el botón rotar del rack para ver la parte trasera (Rear View) desde donde nacen las conexiones.

RACK Designer Next — Guía Técnica Completa

Explicacion tecnica para recrear el proyecto desde cero, con diagramas y conceptos explicados paso a paso.

Tabla de Contenidos

- 1. [Que es este proyecto?](#)
- 2. [Stack Tecnologico \(que usamos y por que\)](#)
- 3. [Arquitectura General](#)
- 4. [El Patron Reactivo \(Proxy ES6\)](#)
- 5. [Estructura de Archivos](#)
- 6. [El Layout de la Interfaz \(CSS Grid\)](#)
- 7. [Cada Area de la UI explicada](#)
- 8. [El Topology View \(Canvas 2D\)](#)
- 9. [Flujo de Datos Completo](#)
- 10. [Como recrear desde cero](#)
- 11. [Diagramas SVG del Proyecto](#)

1. Que es este proyecto?

RACK Designer Next es una aplicacion web que sirve para disenar y administrar centros de datos. Permite:

- Crear **salas** (rooms) que representan habitaciones de un data center
- Crear **gabinetes** (racks) dentro de esas salas
- Colocar **equipos** (servidores, switches, routers, firewalls, UPS, etc.) dentro de los gabinetes
- Conectar los equipos entre si con **cables** (cobre, fibra optica)
- Ver todo en dos vistas: **Vista Física** (como se ve el rack) y **Vista Topologia** (diagrama de red)
- Exportar a **PNG, CSV, Excel** y guardar/cargar proyectos

Lo mas importante: Es una PWA (Progressive Web App) que funciona sin servidor backend. Todos los datos se guardan en el navegador del usuario (localStorage).

Ejecutar el proyecto

Opcion 1: Live Server en VS Code

Click derecho en index.html → "Open with Live Server"

Opcion 2: Cualquier servidor estatico

npx serve .

python -m http.server 8000

No necesitas npm start, webpack, vite ni nada para compilar. Solo abre el index.html.

2. Stack Tecnologico

Que usamos

Tecnologia	Para que sirve	Por que la elegimos
JavaScript ES6+	Toda la logica	Sin frameworks, vanilla puro
HTML5	Estructura de la app	Semanticos, modals, canvas
CSS3	Estilos y layout	Grid, Flexbox, variables CSS
Canvas 2D	Vista de topologia	Animaciones y dibujo de red
Proxy ES6	Estado reactivo	Reemplaza a Redux/useState
localStorage	Guardar datos	Persistencia sin servidor
sessionStorage	Sesiones de auth	Se borra al cerrar pestana
SHA-256	Hasheo de PIN	Seguridad basica de admin
File System API	Abrir/guardar archivos	Sin necesidad de upload
SheetJS	Exportar a Excel	Unica dependencia npm

Que NO usamos (y por que)

No usamos	Por que
React/Vue/Angular	El proyecto es vanilla, sin build step
Webpack/Vite	No hay paso de compilacion
TypeScript	Vanilla JS puro

No usamos	Por que
Backend/API	Todo es local en el navegador
Base de datos	localStorage es suficiente
npm (solo pnpm)	Para instalar marked (script de build)

Diagrama de tecnologías

Ver: doc/doc_img/doc_svg/architecture_overview.svg (seccion Tech Stack)

3. Arquitectura General

El proyecto sigue un patron que se llama "**Reactive Proxy Pattern**". Esto significa que:

- 1. **Toda la informacion vive en un solo lugar** (el store)
- 2. **Cuando algo cambia**, el Proxy lo detecta automaticamente
- 3. **Se guarda** en el navegador (localStorage)
- 4. **Se notifica** a la interfaz que hay datos nuevos
- 5. **La interfaz se redibuja** con los datos actualizados

USUARIO hace algo



EVENT HANDLER ejecuta una funcion



STORE METHOD modifica los datos



PROXY TRAP detecta el cambio



_save() guarda en localStorage



_emit('change') notifica a todos



renderAll() redibuja la interfaz

Ver **diagrama completo**: doc/doc_img/doc_svg/project_flow.svg (Flow 2)

Por que es importante esto?

Porque si intentas modificar el HTML directamente (por ejemplo, document.getElementById('algo').innerHTML = '...'), el proyecto se rompe. **Siempre** debes modificar el store y dejar que la interfaz se actualice sola.

4. El Patron Reactivo

Como funciona el Proxy ES6

// El store crea un Proxy sobre el estado

this._raw = { rooms: [], racks: [], devices: [], ... };

this.state = this._makeProxy(this._raw);

// Cuando haces algo como:

store.state.rooms.push({ id: '1', name: 'Sala A' });

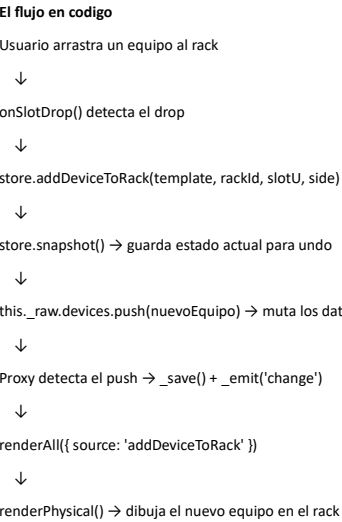
```
// El Proxy detecta:

// 1. Que rooms cambio

// 2. Llama a _save() → guarda en localStorage

// 3. Llama a _emit('change') → notifica a main.js

// 4. main.js llama a renderAll() → redibuja la UI
```



```
renderStats() → actualiza las estadísticas

renderOutliner() → actualiza el arbol

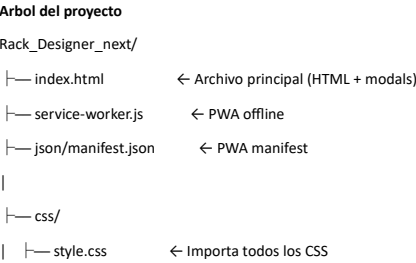
renderBottomPanel() → actualiza la tabla

Ver diagrama: doc/doc_img/doc_svg/project_flow.svg (Flow 3a)

Undo/Redo
```

- Cada vez que mutas el store, snapshot() guarda una copia del estado anterior (maximo 30). Cuando el usuario presiona Ctrl+Z:
1. Se toma el estado actual y se guarda en _redoStack
 2. Se saca el estado anterior de _undoStack
 3. Se sobrescribe _raw con ese estado
 4. Se redibuja todo
- Ver diagrama: doc/doc_img/doc_svg/project_flow.svg (Flow 4)

5. Estructura de Archivos



- | |─ variables.css ← Variables CSS (colores, fuentes, tamanos)
- | |─ layout.css ← Grid layout, header, sidebar, panels
- | └─ components/
 - | |─ panels.css ← Bottom panel, tablas, badges
 - | |─ rack.css ← Tarjetas de rack, flip 3D
 - | |─ faceplates.css ← Faceplates de dispositivos, LEDs
 - | |─ modals.css ← Modales, formularios, acordeon
 - | └─ misc.css ← Iconos SVG, tooltips, drag ghost
- |
- |─ js/
 - | |─ store.js ← ESTADO CENTRAL (Proxy ES6)
 - | |─ main.js ← Orquestador (init, eventos, renderAll)
 - | |─ utils.js ← Funciones auxiliares
 - | |─ icons.js ← SVG inline de todos los dispositivos
 - | |─ demoData.js ← Carga datos de ejemplo
 - | |
 - | |─ auth/
 - | | └─ roles.js ← RBAC (Admin/Editor/Viewer)
 - | |
 - | |─ ui/

- | | |─ catalog.js ← Catalogo de dispositivos + filtros
- | | |─ rack.js ← Renderizado físico de racks
- | | |─ faceplates.js ← HTML de faceplates por tipo
- | | |─ tables.js ← Tabla de inventario/conexiones
- | | |─ outliner.js ← Arbol jerarquico (sala→rack→equipo)
- | | |─ inspector.js ← Panel de propiedades
- | | |─ fileManager.js ← Abrir/guardar archivos (.rack)
- | | |─ modals.js ← Orquestador de modales
- | | |
- | | |─ modals/
 - | | | |─ Globals.js ← Variables globales de modales
 - | | | |─ RackModal.js ← Modal de crear/editar rack
 - | | | |─ DeviceModal.js ← Modal de crear/editar equipo
 - | | | |─ CableModal.js ← Modal de crear/editar cable
 - | | | |─ RoomModal.js ← Modal de crear sala
 - | | | |─ ExportModal.js ← Modal de exportar PNG/CSV/Excel
 - | | | └─ PlacementModal.js ← Modal de colocacion asistida
- | | |
- | | └─ topology/
 - | | |─ TopologyState.js ← Estado del canvas

		└─ TopologyLayout.js	←	Algoritmo de layout automatico
		└─ TopologyRenderer.js	←	Dibujado Canvas 2D
		└─ TopologyEvents.js	←	Interacciones del canvas
		└─ TopologyOrchestrator.js	←	Lifecycle del canvas
		└─ xlsx.full.min.js	←	SheetJS (exportar Excel)
		└─ mobile-drag-drop.min.js	←	Polyfill touch drag
		└─ mobile-drag-drop-scroll.min.js		
		└─ assets/icons/	←	Iconos SVG por tipo de equipo
		└─ doc/	←	Documentacion
		└─ .agents/	←	Perfiles de agentes IA
		└─ .py/	←	Scripts Python utilitarios

Ver diagrama: doc/doc_img/doc_svg/architecture_overview.svg (File Structure)

Orden de carga de CSS

variables.css → layout.css → panels.css → rack.css → faceplates.css → modals.css → misc.css

Este orden es **importante** porque CSS se resuelve en cascada. Si misc.css define algo que layout.css tambien define, gana misc.css (se carga despues).

Scripts en index.html

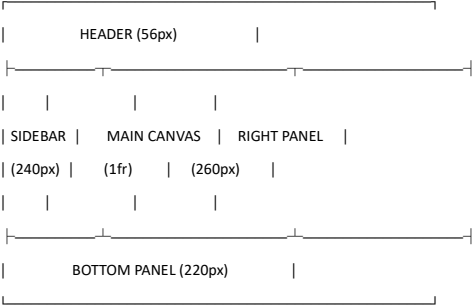
Todos los <script> van al final del <body>, en orden:

- 1. Vendor: xlsx.full.min.js, mobile-drag-drop.min.js
- 2. Core: store.js, utils.js, icons.js, demoData.js

- 3. Auth: auth/roles.js
- 4. UI: ui/catalog.js, ui/tables.js, ui/outliner.js, ui/inspector.js
- 5. Modals: todos los archivos en ui/modals/
- 6. Topology: todos los archivos en ui/topology/
- 7. Orchestration: ui/modals.js, ui/fileManager.js
- 8. Init: main.js (ultimo, porque depende de todo lo demas)

6. El Layout de la Interfaz (CSS Grid)

La app usa **CSS Grid** para dividir la pantalla en 5 areas fijas:



CSS que define esto

#app {

```
display: grid;

grid-template-rows: 56px 1fr 220px;

grid-template-columns: 240px 1fr 260px;

grid-template-areas:

    "header header    header"

    "sidebar main      right-panel"

    "bottom bottom    bottom";

height: 100vh;

overflow: hidden;

}
```

Variables CSS importantes

```
:root {

    /* Dimensiones */

    --sidebar-w: 240px;

    --right-panel-w: 260px;

    --header-h: 56px;

    --bottom-h: 220px;

    --rack-unit-h: 24px;    /* 1U = 24px */

    /* Colores oscuros */

    --bg-main: #0b0f19;    /* Fondo principal */
```

```
--bg-card: #151c2e;    /* Tarjetas */

--border: #25304b;    /* Bordes */

--accent: #0ea5e9;    /* Color de acento (cian) */

    /* Fuentes */

    --font-ui: 'Outfit', sans-serif;

    --font-mono: 'JetBrains Mono', monospace;

}
```

Ver diagrama: doc/doc_img/doc_svg/layout_header.svg, layout_sidebar.svg, etc.

7. Cada Area de la UI explicada

Header (56px)

Archivo: index.html (seccion #header), estilos en css/layout.css

Contiene:

- Logo + nombre del proyecto
- Menu del proyecto (abrir, guardar, tema, demo)
- Dropdown de salas (selector de habitacion)
- Dropdown de racks
- Tabs de vista (Fisica / Topologia)
- Buscador global
- Botones Undo/Redo

- Badge de usuario (rol)

Ver: doc/doc_img/doc_svg/layout_header.svg

Sidebar (240px)

Archivo: index.html (seccion #sidebar), renderizado por js/ui/catalog.js

Contiene:

- Botones "+ Rack" y "+ Equipo"
- Buscador del catalogo
- Filtros por categoria (Todos, Servidor, Red, Storage, Cableado, Energia, Accesorios, Piso)
- Lista de dispositivos del catalogo (21 plantillas predefinidas)
- Cada item es arrastrable al rack (drag & drop)

Ver: doc/doc_img/doc_svg/layout_sidebar.svg

Main Canvas (1fr)

Archivo: index.html (seccion #main), renderizado por js/ui/rack.js

Contiene dos vistas:

- **Vista Fisica:** Tarjetas de rack con faceplates de dispositivos, vista frontal/trasera (flip 3D), dispositivos de piso
- **Vista Topologia:** Canvas 2D con diagrama de red animado

Ver: doc/doc_img/doc_svg/layout_main_canvas.svg

Right Panel (260px)

Archivo: index.html (seccion #right-panel)

Tres secciones:

1. **Outliner:** Arbol jerarquico (Sala → Rack → Equipo)

2. **Propiedades (Inspector):** Detalles del elemento seleccionado

3. **Estadisticas:** Metricas globales (salas, racks, equipos, conexiones)

Ver: doc/doc_img/doc_svg/layout_right_panel.svg

Bottom Panel (220px)

Archivo: index.html (seccion #bottom), renderizado por js/ui/tables.js

Contiene:

- Tabs: Inventario / Conexiones
- Tabla con todas las propiedades de los dispositivos
- Edicion inline de celdas (IP, MAC, nombre, etc.)
- Exportar a CSV/Excel
- Boton expandir/contraer

Ver: doc/doc_img/doc_svg/layout_bottom_panel.svg

8. El Topology View (Canvas 2D)

La vista de topologia es un lienzon HTML5 Canvas que dibuja:

Colores oficiales del canvas

Elemento	Color	Codigo
Fondo	Azul	#2255aa
Grid puntos	Blanco 13% opacity	#ffffff22

Elemento	Color	Codigo
Salas	Naranja	#f97316
Bordes sala activa	Blanco 4px	#ffffff
Bordes sala inactiva	Naranja oscuro 3px	#c2410c
Racks	Verde	#65a30d
Nodos (card)	Navaja oscuro	#0f172a
Borde nodo	Color del tipo	TYPE_COLORS[dev.type]
Conexiones	Color del cable	conn.color
Particulas	Blanco + glow	#ffffff + shadow

Como se dibuja

drawTopo() se ejecuta 60 veces por segundo

- ↓
1. Limpia el canvas (fillRect azul)
- ↓
2. Dibuja grid de puntos (40px spacing)
- ↓
3. Dibuja salas (roundRect naranja)
- ↓

4. Dibuja racks dentro de salas (roundRect verde)



5. Dibuja conexiones (curvas Bezier cubicas)



6. Dibuja particulas animadas (4 por cable)



7. Dibuja nodos de dispositivos (card o circle)



8. Dibuja HUD de hover (tooltip)

Layout automatico

El algoritmo en TopologyLayout.js organiza:

1. **Salas** horizontalmente con 60px de espacio entre ellas
2. **Racks** horizontalmente dentro de cada sala
3. **Equipos** verticalmente apilados dentro de cada rack
4. **Equipos de piso** en grilla debajo de los racks (max 4 columnas)

Interacciones

- **Arrastrar nodo:** Mueve el equipo en el diagrama
- **Arrastrar rack:** Mueve todo el rack con sus equipos
- **Arrastrar sala:** Mueve la sala completa
- **Resize handle:** Cambia el tamaño de sala/rack
- **Rueda del mouse:** Zoom in/out

- **Click medio + arrastrar:** Pan (mover vista)
- **Doble click nodo:** Abre modal de crear cable
- **Doble click cable:** Abre modal de editar cable

Ver: doc/doc_img/doc_svg/layout_topology.svg

9. Flujo de Datos Completo

Crear un dispositivo (Drag & Drop)

1. Usuario arrastra item del catalogo
2. onCatalogDragStart() guarda el template
3. Usuario suelta en un slot del rack
4. onSlotDrop() obtiene el template
5. store.addDeviceToRack() es llamado
6. snapshot() guarda estado para undo
7. Se crea el objeto device con uid()
8. Se hace push a this._raw.devices
9. Proxy detecta el push
10. _save() guarda en localStorage
11. _emit('change', { source: 'addDeviceToRack' })
12. renderAll() redibuja:
 - renderPhysical() → nuevo faceplate aparece
 - renderStats() → estadísticas se actualizan

- renderOutliner() → arbol se actualiza
- renderBottomPanel() → tabla se actualiza

Ver: doc/doc_img/doc_svg/project_flow.svg (Flow 3a)

Crear una conexion (cable)

1. Click derecho en equipo → "Agregar Cable"
 2. Se abre CableModal
 3. Usuario selecciona: origen, destino, puertos, tipo, color
 4. Click en "Conectar"
 5. store.addConnection() es llamado
 6. Se crea el objeto connection
 7. Proxy → _save() → _emit('change')
 8. renderPhysical() → puertos iluminados en faceplate
 9. Si esta en Topologia: drawTopo() dibuja curva Bezier
- Ver: doc/doc_img/doc_svg/project_flow.svg (Flow 3b)

Guardar proyecto

1. usuario click "Guardar"
2. fileManager.saveProject()
3. Verifica si File System API esta disponible
4. Si SI: showSaveFilePicker() → writeToFile()
5. Si NO: downloadJSON() → crea blob → descarga
6. Formato: { version: 1, project: store._raw, catalog: CATALOG }

7. Extension: .rack (es JSON)

Cargar proyecto

- 1. usuario click "Abrir"
- 2. fileManager.openProject()
- 3. Verifica File System API
- 4. Si SI: showOpenFilePicker() → getFile()
- 5. Si NO: click en input file → FileReader
- 6. JSON.parse(text) → extrae project y catalog
- 7. store.loadData(data.project) → sobrescribe estado
- 8. C undo/redo se limpian
- 9. renderAll() → redibuja todo

10. Como recrear desde cero

Si quieres construir algo similar paso a paso, aqui tienes el orden:

Paso 1: Crear la estructura basica

```
mkdir rack-designer
cd rack-designer
touch index.html css/style.css js/store.js js/main.js
```

Paso 2: Definir el layout CSS

Crea css/variables.css con las variables CSS basics:

```
:root {
```

```
--sidebar-w: 240px;

--right-panel-w: 260px;

--header-h: 56px;

--bottom-h: 220px;

--bg-main: #0b0f19;

--bg-card: #151c2e;

--border: #25304b;

--accent: #0ea5e9;

--font-ui: 'Outfit', sans-serif;

--font-mono: 'JetBrains Mono', monospace;

}
```

Crea css/layout.css con el Grid:

```
* { margin: 0; padding: 0; box-sizing: border-box; }

body { font-family: var(--font-ui); background: var(--bg-main); color: #e2e8f0; }
```

```
#app {

  display: grid;

  grid-template-rows: var(--header-h) 1fr var(--bottom-h);

  grid-template-columns: var(--sidebar-w) 1fr var(--right-panel-w);

  grid-template-areas:

    "header header header"
```

"sidebar main right-panel"

"bottom bottom bottom";

height: 100vh;

}

Paso 3: Crear el store reactivo

El corazon del proyecto. Crea js/store.js:

```
class Store {

  constructor() {

    this._listeners = {};

    this._undoStack = [];

    this._redoStack = [];

    this._raw = this._load();

    this.state = this._makeProxy(this._raw);

  }
```

```
_makeProxy(obj) {

  const store = this;

  return new Proxy(obj, {

    set(target, key, value) {

      target[key] = value;

      store._save();

    }
```

store._emit('change', { key, value });

return true;

}

});

}

_load() {

const saved = localStorage.getItem('RACK_DESIGNER_NEXT_STATE');

if (saved) return JSON.parse(saved);

return {

rooms: [{ id: '1', name: 'Data Center' }],

racks: [],

devices: [],

connections: [],

currentRoomId: '1'

};

}

_save() {

localStorage.setItem('RACK_DESIGNER_NEXT_STATE', JSON.stringify(this._raw));

}

```
on(event, cb) {
  if (!this._listeners[event]) this._listeners[event] = [];
  this._listeners[event].push(cb);
}

_emit(event, data) {
  (this._listeners[event] || []).forEach(cb => cb(data));
}

snapshot() {
  this._undoStack.push(JSON.parse(JSON.stringify(this._raw)));
  if (this._undoStack.length > 30) this._undoStack.shift();
  this._redoStack = [];
}

// ... mas metodos (addRoom, addRack, addDevice, etc.)
}
```

```
window.store = new Store();
```

Paso 4: Crear el orquestador

Crea js/main.js:

```
function renderAll() {
  renderRoomTabs();
  renderRackSelector();
  renderStats();
  renderPhysical();
  renderOutliner();
  renderBottomPanel();
}
```

// Cuando el store cambia, redibujar todo

```
store.on('change', renderAll);
```

// Render inicial

```
document.addEventListener('DOMContentLoaded', () => {
  renderAll();
});
```

Paso 5: Crear los componentes UI

Crea archivos en js/ui/:

- catalog.js → Lista de dispositivos arrastrables
- rack.js → Renderizado de racks con drag & drop

- faceplates.js → HTML de cada tipo de dispositivo
- tables.js → Tabla de inventario
- outliner.js → Arbol jerarquico
- inspector.js → Panel de propiedades

Paso 6: Crear los modales

Crea archivos en js/ui/modals/:

- RackModal.js → Crear/editar rack
- DeviceModal.js → Crear/editar equipo
- CableModal.js → Crear/editar cable
- RoomModal.js → Crear sala

Paso 7: Agregar Canvas 2D (Topología)

Crea js/ui/topology/:

- TopologyState.js → Estado del canvas
- TopologyRenderer.js → Dibujado con Canvas 2D
- TopologyEvents.js → Mouse events (drag, zoom, pan)
- TopologyLayout.js → Layout automatico
- TopologyOrchestrator.js → Lifecycle

Paso 8: Agregar autenticacion

Crea js/auth/roles.js con SHA-256 y sessionStorage.

Paso 9: Agregar PWA

Crea service-worker.js para cache offline y json/manifest.json.

11. Diagramas SVG del Proyecto

Todos los diagramas estan en doc/doc_img/doc_svg/:

Archivo	Descripcion
layout_header.svg	Layout detallado del header (1100x140)
layout_sidebar.svg	Layout detallado del sidebar con catalogo (360x780)
layout_main_canvas.svg	Layout del area principal con racks (720x580)
layout_right_panel.svg	Layout del panel derecho (340x680)
layout_bottom_panel.svg	Layout del panel inferior con tabla (1100x360)
layout_topology.svg	Vista de topología con salas, racks, conexiones (1400x700)
architecture_overview.svg	Arquitectura completa del proyecto (2400x2800)
architecture_current.svg	Diagrama de la arquitectura actual (1200x1650) con flujos reactivos, seguridad RBAC y fallbacks offline
file_communication_flow.svg	Diagrama de comunicación y llamadas inter-módulo (1400x1250)
project_flow.svg	Flujo de datos completo (2000x2600)

Resumen Final

El proyecto RACK Designer Next es una app web vanilla que:

1. **No necesita build step** → Solo abre index.html

- 2. Usa Proxy ES6 para estado reactivo (como Redux pero nativo)
- 3. Guarda todo en localStorage (sin backend)
- 4. Tiene 5 areas fijas en CSS Grid (header, sidebar, main, right, bottom)
- 5. Canvas 2D para la vista de topologia
- 6. Drag & drop nativo para colocar equipos
- 7. 3 roles (Admin, Editor, Viewer) con SHA-256
- 8. Exporta a PNG, CSV, Excel y JSON

La regla mas importante: Nunca mutar el DOM directamente. Siempre actualizar el store y dejar que renderAll() se encargue de redibujar.

9. Renderizado de Cableado Físico

A diferencia de la topología (Canvas), el cableado físico 2D se resuelve inyectando dinámicamente un lienzo

- Los componentes de faceplates.js inyectan anclajes data-device-id y data-port al DOM.
- drawPhysicalCables() (en ack.js) procesa las conexiones y traza trayectorias ortogonales bordeando las tarjetas de rack de manera reactiva.

Dimensiones y Medidas del Layout de la Interfaz

Este documento centraliza y detalla el esquema de rejilla (CSS Grid), anchos, alturas y reglas de diseño que componen la interfaz de usuario de **RACK Designer Next**.

1. Rejilla Estructural Principal (CSS Grid)

El contenedor principal del sistema (#app) utiliza un diseño bidimensional implementado con CSS Grid, dividido en 3 filas y 3 columnas:

```
#app {
  display: grid;
```

```
grid-template-rows: var(--header-h) minmax(0, 1fr) auto;
```

```
grid-template-columns: var(--sidebar-w) 1fr var(--right-panel-w);
```

```
grid-template-areas:
```

```
"header header header"
```

```
"sidebar main right-panel"
```

```
"bottom bottom bottom";
```

```
height: 100vh;
```

```
overflow: hidden;
```

```
}
```

📏 Tabla de Dimensiones de Rejilla (Escritorio)

Elemento	Área Grid	Ancho	Alto	Comportamiento
Cabecera (#header)	header	100%	56px (--header-h)	Fijo en la parte superior.
Barra Lateral (#sidebar)	sidebar	50px (--sidebar-w)	Flex	Fijo a la izquierda, contiene iconos de categoría.
Lienzo Central (#main)	main	1fr (Restante)	Flex (minmax(0, 1fr))	Contenedor principal de vistas (Física/Topología).
Panel Derecho (#right-panel)	right-panel	300px (--right-panel-w)	Flex	Contiene el Outliner, Inspector y Estadísticas.
Panel Inferior (#bottom)	bottom	100%	220px (--bottom-h)	Pestañas de inventario/conexiones. Colapsable.

2. Dimensiones Detalladas por Componente

⚡ HEADER (Cabecera)

- **Altura Total:** 56px (--header-h).
- **Área de Logo (.header-logo-area):** Ancho fijo de 260px, alineado a la izquierda con borde derecho separator de 1px.
- **Área Principal (.header-main-area):** flex: 1 para ocupar el espacio restante. Contiene las pestañas de sala, racks y controles globales.

BARRA LATERAL & CATÁLOGO FLOTANTE

- **Barra de Iconos (#sidebar):**
 - Ancho: 50px.
 - Alineación: justify-self: start.
 - Z-Index: 10 (por debajo del catálogo flotante).
- **Catálogo Flotante (#catalog-flyout):**
 - Ubicación: Comparte el área grid sidebar.
 - Ancho: 210px.
 - Alineación: justify-self: end.
 - Z-Index: 50 (flota sobre el lienzo central #main extendiéndose hacia la derecha de la barra lateral).

LIENZO CENTRAL (#main)

- **Grid de Fondo (Dotgrid):** Renderizado con un gradiente radial con puntos (var(--grid-dot)) de 1px espaciados en una cuadrícula de **28px × 28px** con opacidad de 0.4.

PANEL DERECHO (#right-panel)

El panel derecho se subdivide en tres secciones organizadas verticalmente con flex-direction: column:

1. **Outliner (#outliner-section):** flex: 0 1 auto con una altura máxima restringida a **25%** para evitar el desplazamiento excesivo.
2. **Inspector (#inspector-section):** flex: 1 para absorber dinámicamente todo el espacio vertical sobrante.
3. **Estadísticas (#stats-section):** Altura fija con flex: 0 0 170px en la parte inferior del panel.

PANEL INFERIOR (#bottom)

- **Altura Expandido:** 220px (--bottom-h).
- **Altura Colapsado (Clase .collapsed):** Reducido estrictamente a **38px** (solo visible la barra de pestañas).
- **Transiciones:** Animado suavemente mediante cambios de altura y transformaciones CSS.

3. Normalización Matemática Estricta

Para mantener consistencia estética de acuerdo al Design System, todos los controles interactivos básicos de la interfaz tienen dimensiones unificadas por CSS:

- **Altura Única de Controles: 24px** de altura fija con box-sizing: border-box.
- **Elementos Afectados:**
 - Botones principales y secundarios (.btn-primary, .btn-secondary, .btn-cancel, .btn-confirm).
 - Pestañas de control (.tab-pill, .view-tab, .filter-tab, .room-tab).
 - Botones de cabecera e iconos de barra superior (.h-btn).
 - Entradas de texto, números, contraseñas y selectores (input[type="text"], input[type="number"], input[type="password"], select).

4. Adaptabilidad y Responsividad (Breakpoints)

Vista de Tableta (max-width: 1024px)

- **Estructura Grid:** Se reconfigura a dos columnas principales: grid-template-columns: 60px 1fr;
- **Ocultación:** El panel derecho (#right-panel) se oculta y la barra lateral de iconos pasa de 50px a 60px de ancho.
- **Simplificación:** Textos secundarios y píldoras de capacidad detalladas del catálogo se ocultan para maximizar la legibilidad.

Vista de Móvil (max-width: 768px)

- **Estructura Grid:** Se aplan a una sola columna vertical:
- #app {
- grid-template-columns: 1fr;
- grid-template-rows: auto 1fr auto;
- grid-template-areas:
- "header"
- "main"
- "bottom";
- }
- **Cabecera:** Se expande verticalmente (height: auto). La marca superior y los controles principales se dividen en dos líneas con scroll horizontal independiente para evitar desbordes.
- **Barra Lateral Móvil (Menú Desplegable):**
 - Se comporta como un cajón flotante (position: fixed) sobre toda la pantalla.
 - Ancho: **85%** del viewport (con un máximo de 320px).
 - Desplazamiento: Animación de deslizamiento de izquierda a derecha (left: -100% a left: 0) mediante la clase .open.
 - Capa de oscurecimiento trasera (#mobile-overlay) con z-index 9998.
- **Panel Inferior Fullscreen:** Al hacer clic en "Expandir", el panel inferior toma el **85vh** de la pantalla para permitir la visualización cómoda de las tablas de datos en pantallas táctiles.

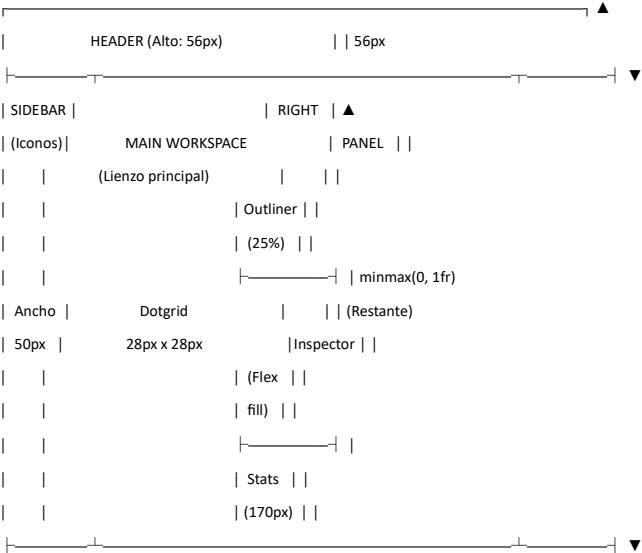
5. Diagramas del Layout

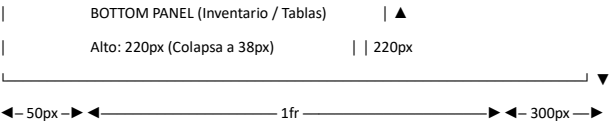
 Estructura del Grid Principal (Mermaid)

No diagram type detected matching given configuration for text: grid

header:3 header:3 header:3
sidebar:1 main:1 right-panel:1
bottom:3 bottom:3 bottom:3

 Esquema Visual de Proporciones (Píxeles)





Dimensiones y Medidas de la Cabecera (Header Layout)

Este documento detalla el esquema de layout, dimensiones de componentes, cajas CSS y comportamiento responsivo de la cabecera principal (#header) de **RACK Designer Next**.

1. Estructura General y Dimensiones Base

La cabecera funciona como la barra de navegación y control principal de la aplicación. Está posicionada estáticamente en el área de rejilla header con un comportamiento flexbox horizontal en pantallas de escritorio:

```
#header {
  grid-area: header;

  display: flex;

  align-items: center;

  padding: 0;

  background: var(--bg-card); /* Color oscuro translúcido */

  border-bottom: 1px solid var(--border);

  z-index: 100;

  height: var(--header-h); /* 56px */
}
```

División Estructurada

La cabecera se divide horizontalmente en dos contenedores principales:

- Área del Logo (.header-logo-area):** Contiene el logotipo, nombre del proyecto, archivo abierto y el menú lateral del proyecto.
- Área Principal de Controles (.header-main-area):** Agrupa los selectores de salas/racks, cambio de vista física/topológica, buscador global, historial (deshacer/rehacer) y estados de conexión.

2. Dimensiones Detalladas de Componentes

📁 Área del Logo (.header-logo-area)

- Ancho Fijo: 260px** (flex-shrink: 0).
- Altura:** 100% (56px).
- Margen y Relleno:** padding: 0 16px.
- Componentes Internos:**
 - Botón de Menú Móvil (#mobile-menu-btn):** display: none en escritorio. Dimensiones normalizadas de 24px × 24px.
 - Imagen del Logo (.logo-icon):** Medida fija de 24px × 24px (object-fit: contain) con filtro de resplandor (drop-shadow de 8px var(--accent)).
 - Texto Subtítulo (.logo-sub o #project-filename):** Tamaño de fuente --text-xs (10px), fuente monoespaciada y margen superior de -4px.
 - Botón del Menú del Proyecto (#btn-project-menu):** Dimensiones de 24px × 24px con transiciones de color en hover.

⚙️ Área Principal (.header-main-area)

- Ancho:** Flexible (flex: 1, min-width: 0).
- Margen y Relleno:** padding: 0 16px con espacio de separación uniforme (gap: 12px).
- Componentes Internos (Altura Estricta de 24px):**
 - Selectores Desplegables (Salas/Racks):**

- Ancho mínimo de la etiqueta interna: Ajustable según el texto.
 - Relleno interno del botón: 0 10px con iconos de 14px (origen) y 12px (flecha de despliegue).
 - Botón para Añadir (.nav-btn-add): Dimensiones fijas de 24px × 24px.
- **Divisor Horizontal (.h-divider):** Ancho de 1px y altura de **32px** (color --border).
- **Pestañas de Vista (.view-tabs):**
 - Contenedor: Borde de 1px y bordes redondeados (var(--radius) = 6px).
 - Botón de Vista (.view-tab): Relleno de 5px 14px en cada pestaña.
- **Buscador Global (.h-search):**
 - Ancho en Reposo: **200px** (padding de entrada: 4px 10px 4px 30px).
 - Ancho en Foco: **240px** (transición fluida de 0.2s).
- **Grupo de Historial (.h-btn-group):**
 - Botones individuales (#btn-undo, #btn-redo): Normalizados a 24px de alto, con bordes redondeados solo en esquinas exteriores (Izquierdo: 6px 0 0 6px, Derecho: 0 6px 6px 0).
- **Punto de Estado (.status-dot):** Círculo de 7px × 7px con animación de pulso y resplandor de color verde (#10b981).
- **Insignia de Usuario (#user-badge):** Relleno de 2px 10px con bordes completamente redondeados (10px).

3. Comportamiento en Vistas Móviles (max-width: 768px)

En pantallas móviles, el espacio horizontal es limitado, por lo que la cabecera reestructura su layout mediante flexbox vertical:

- **Flex-Direction:** Cambia a column con altura automática (height: auto) y desborde visible (overflow: visible).
- **Línea 1 (Logo & Menú):**
 - .header-logo-area toma el **100%** de ancho.

- El botón del menú móvil (#mobile-menu-btn) pasa a display: flex.
 - El borde derecho separator se elimina y se añade un borde inferior de 1px.
- **Línea 2 (Barra de Navegación):**
 - .header-main-area se convierte en una fila horizontal con scroll (overflow-x: auto) sin ajustar líneas (flex-wrap: nowrap).
 - El espaciador central (.spacer) se oculta (display: none).
 - Todos los hijos directos se bloquean para evitar el encogimiento (flex-shrink: 0).

4. Diagramas de Cabecera

📱 Composición Horizontal (Escritorio)

#header (Alto: 56px)

.header-main-area (Flexible: 1fr)

Salas / Dropdown

Racks / Dropdown

Divisor (32px alto)

Pestañas de Vista

Espaciador (Flex: 1)

Buscador (200px -> 240px)

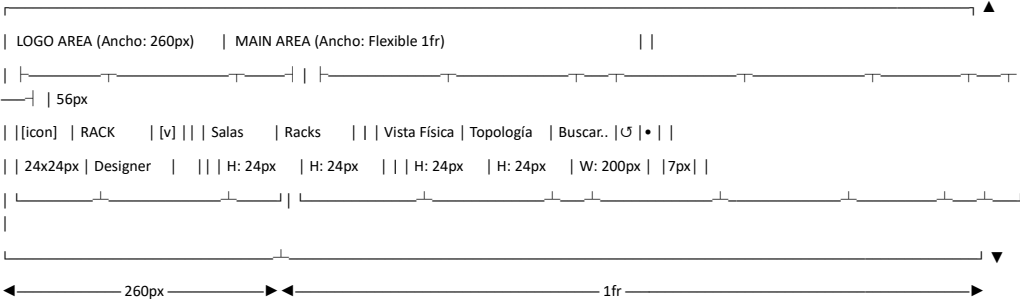
Undo/Redo (24px alto)

Punto Estado (7x7)

.header-logo-area (Ancho: 260px)

Logo Icon (24x24)

 Distribución de Píxeles de Cabecera



Dimensiones y Medidas del Panel Derecho (Right Panel Layout)

Este documento detalla el esquema de layout, dimensiones de los componentes, lógica de colapso y comportamiento responsivo del panel lateral derecho (#right-panel) de **RACK Designer Next**.

1. Estructura General y Dimensiones Base

El panel derecho está posicionado en el área de rejilla right-panel y funciona como el centro de información contextual del proyecto, organizando de forma vertical tres bloques: el árbol de elementos (Outliner), el inspector de propiedades y las estadísticas globales.

```
#right-panel {  
  
  grid-area: right-panel;  
  
  display: flex;  
  
  flex-direction: column;  
  
  width: 300px; /* Ancho fijo (var(--right-panel-w)) */  
  
  background: var(--bg-card);  
  
  border-left: 1px solid var(--border);  
  
  z-index: 90;  
  
  overflow: hidden;  
  
}
```

2. Distribución Interna (Comportamiento Flex)

Los tres paneles internos se distribuyen de forma vertical mediante Flexbox. Cada sección (.right-panel-section) tiene una lógica de flexibilidad distinta:

-  **1. Outliner (#outliner-section)**
 - Comportamiento Flex:** flex: 0 1 auto (no se expande por defecto, pero puede encogerse).
 - Altura Máxima:** Limitada estrictamente a **25%** (max-height: 25%) con desborde interno (overflow-y: auto) para asegurar que el Inspector tenga espacio suficiente.
 - Transiciones:** Animación en max-height y flex de 0.2s ease al contraer/expandir.
-  **2. Inspector de Propiedades (#inspector-section)**
 - Comportamiento Flex:** flex: 1 (ocupa de forma fluida todo el espacio restante disponible entre el Outliner y las Estadísticas).
 - Contenido:** Relleno de padding: 12px y tamaño de fuente de 13px (--text-md) para etiquetas de propiedades de hardware.

3. Estadísticas (#stats-section)

- Comportamiento Flex: flex: 0 0 170px (altura fija de 170px, no se deforma ni se encoge ante cambios de resolución vertical).
- Contenido: Muestra las métricas globales del proyecto (salas, racks, consumo de energía y conexiones). Relleno interno de padding: 12px.

3. Lógica de Colapso Dinámico

Las secciones del Outliner y Estadísticas permiten contraerse haciendo doble clic o clic en su cabecera para liberar espacio para el Inspector de propiedades:

- Clase de Colapso (.collapsed):
 - Dimensiones: El flex se bloquea a tamaño automático mínimo: flex: 0 0 auto !important;
 - Ocultación de Contenido: Todos los elementos hijos directos (excepto la cabecera) se ocultan: display: none !important;
- Icono Indicador (.toggle-icon):
 - Fila normal: Flecha hacia abajo ▼.
 - Fila colapsada: Rota -90 grados convirtiéndose en ► (transform: rotate(-90deg)).

4. Medidas de Micro-Interfaz (Outliner Tree)

El árbol de nodos del Outliner sigue una estructura jerárquica con medidas estrictas de alineación:

- Cabeceras de Nodos (.outliner-summary):
 - Relleno: padding: 4px 4px.
 - Bordes: Radio de curvatura de 4px para el hover.
 - Fuente: Tamaño 13px (--text-md) y grosor 500 (semibold).
 - Caret Indicador (::before): Ancho fijo de 12px, tamaño de fuente 9px. Rota 90 grados al abrir el <details> nativo.
- Tarjetas de Dispositivos (.outliner-device):

- Relleno: padding: 3px 4px.
- Fuente: Tamaño de letra reducido a 12px (--text-base).
- Iconos de Nodos (.outliner-node-icon):
 - Iconos de Sala/Rack: 13px × 13px con opacidades del 100% (sala) y 85% (rack) y color acentuado (var(--accent)).
 - Icono de Dispositivo: 11px × 11px con opacidad del 70%.

5. Adaptabilidad y Breakpoints

Debido a que los datos contextuales son secundarios respecto al lienzo principal, el panel derecho se descarta en pantallas estrechas:

- Vista de Tableta (max-width: 1024px):
 - El grid del #app se reduce a dos columnas: 60px 1fr (Sidebar e Workspace).
 - El panel derecho se oculta completamente de la visualización al no estar definido en las columnas y áreas del grid.
- Vista de Móvil (max-width: 768px):
 - El grid cambia a una columna vertical. El área right-panel se elimina de la definición grid-template-areas, ocultando el panel de forma absoluta.

6. Diagramas del Panel Derecho

Distribución Flex Vertical (Escritorio)

#right-panel (Ancho: 300px)

Outliner (#outliner-section)

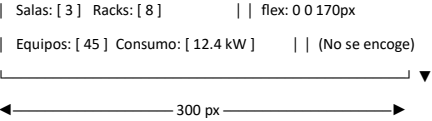
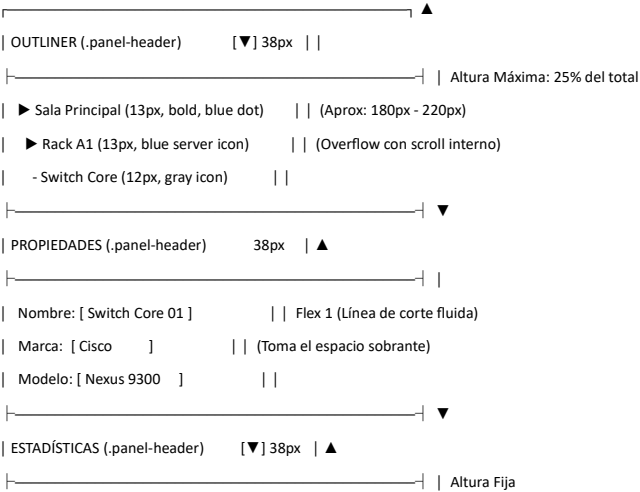
Flex: 0 1 auto (Max 25%)

[Árbol jerárquico de salas y racks]

Inspector (#inspector-section)
Flex: 1 (Rellena restante)
[Detalles y edición de propiedades]

Estadísticas (#stats-section)
Flex: 0 0 170px (Fijo)
[Métricas del Data Center]

Esquema de Alturas en Píxeles



Dimensiones y Medidas del Sidebar y Catálogo Flotante

Este documento detalla el esquema de layout, dimensiones de los componentes, animaciones y comportamiento responsivo de la barra lateral (#sidebar) y el panel del catálogo flotante (.catalog-flyout) de **RACK Designer Next**.

1. Barra Lateral de Iconos (#sidebar)

El sidebar es una barra compacta vertical alineada a la izquierda que contiene los accesos rápidos a las distintas categorías del catálogo.

```
#sidebar {  
  display: flex;  
  
  flex-direction: column;  
  
  width: 50px; /* Ancho fijo */  
  
  grid-area: sidebar;  
  
  justify-self: start;  
  
  background: var(--bg-card);  
  
  border-right: 1px solid var(--border);  
  
  z-index: 10;  
}
```

Componentes de la Barra Lateral

- Contenedor de Iconos (.sidebar-icons):
 - Estructura: Flex vertical (flex-direction: column) con espaciado de gap: 6px.
 - Espaciado: Relleno vertical de padding: 10px 0.
 - Scroll: Desbordamiento vertical oculto (overflow-y: auto), ocultando la barra de scroll nativa del navegador.
- Botón de Categoría (.sb-category-btn):
 - Dimensiones: Fijo 32px x 32px (flex-shrink: 0).
 - Bordes: Radio de curvatura de 6px (var(--radius)).
 - Estados:
 - Activo (.active): Fondo var(--bg-card2) y contorno destacado (border-color: var(--accent)).
 - Hover: Fondo ligeramente translúcido (rgba(255, 255, 255, 0.05)) y texto principal en alto contraste.

2. Catálogo Flotante (#catalog-flyout)

El catálogo flotante funciona como una persiana deslizante que se expande hacia la derecha al seleccionar una categoría, solapándose sobre el área de trabajo #main.

```
.catalog-flyout {
  grid-area: sidebar;
  justify-self: end;
  width: 210px; /* Ancho fijo flotante */
  height: 100%;
  background: var(--bg-card);
```

```
border-right: 1px solid var(--border);

z-index: 50; /* Por encima de #sidebar y #main */

display: flex;
flex-direction: column;
}
```

Estructura de Control del Catálogo

- Cabecera (.flyout-header): Relleno interno de 12px 14px, con borde inferior separator. Contiene el título del catálogo y el botón de cierre (cruz normalizada a control de 24px).
- Acciones Rápidas (.flyout-actions):
 - Layout: Grid de dos columnas (1fr 1fr) con espacio de gap: 6px y relleno de 10px 14px.
 - Botones (+ Rack y + Equipo): Altura estricta normalizada a 24px.
- Buscador Interno (.flyout-search):
 - Espaciado: Relleno de 8px 14px con borde inferior.
 - Campo de Entrada (input): Altura normalizada a 24px y fuente monoespaciada para concordar con la cabecera.

3. Tarjetas de Equipos (.catalog-item)

Las plantillas de dispositivos y racks listadas en la sección .catalog se presentan como tarjetas arrastrables (drag & drop):

- Contenedor Principal (.catalog-item):
 - Estructura: Flex horizontal (align-items: center) con separación de gap: 6px.
 - Dimensiones de Relleno: padding: 8px 8px, margen inferior de 6px.
 - Bordes: Radio de curvatura de 6px (var(--radius)).

- Fondo: Color de tarjeta secundario (var(--bg-card2)).
- Estados:**
 - Hover:** Borde destacado (border-color: var(--accent)), fondo iluminado (var(--accent-glow)) y animación de desplazamiento sutil de **2px hacia la derecha** (transform: translateX(2px)).
 - Dragging (.dragging):** Opacidad reducida a 0.4 e icono de cursor en grabbing.
- Icono de Tarjeta (.cat-icon):** Dimensiones fijas de **32px × 32px** con bordes redondeados (6px) y centrado flexible de iconos vectoriales.
- Información de Tarjeta (.cat-info):** Contenedor flexible (flex: 1, min-width: 0) para evitar desbordes de texto.
 - Nombre (.cat-name):** Tamaño de fuente --text-sm (11px), fuente semibold (60px) y límite estricto de dos líneas de texto (-webkit-line-clamp: 2).
 - Metadatos (.cat-meta):** Tamaño --text-xs (10px), fuente monoespaciada, color atenuado (--text-muted) y corte de desborde elíptico (ellipsis).

4. Adaptabilidad y Breakpoints

 Vista de Tableta (max-width: 1024px)

- Barra de Iconos:** El área de la columna de sidebar de #app se incrementa a **60px** de ancho.
- Simplificación Extrema:** Se ocultan las secciones de búsquedas (.sb-search), pestañas de filtros (.sb-filter-tabs), información textual de tarjetas (.cat-info) y etiquetas de tamaño.
- Rediseño Visual:** Las tarjetas de catálogo (.catalog-item) se centran (justify-content: center) y el contenedor del icono (.cat-icon) se agranda a **40px × 40px** para facilitar el arrastre táctil en pantallas medianas.

 Vista de Móvil (max-width: 768px)

- Rejilla:** El sidebar se extrae de la rejilla de #app (que pasa a ser de 1 columna).
- Panel Desplegable (Cajón Lateral):**
 - Se comporta como un panel flotante de pantalla completa (position: fixed, height: 100vh).
 - Ancho: **85%** de la pantalla, acotado a un máximo de **320px**.

- Desplazamiento: Se oculta a la izquierda (left: -100%) y se desliza mediante animación CSS al añadir la clase .open (left: 0).
- Z-Index: Elevado a **9999**, acompañado de un fondo atenuador (#mobile-overlay) con z-index 9998 para bloquear interacciones del lienzo trasero.

5. Diagramas del Layout

 Despliegue del Catálogo Flotante (Escritorio)

Área Grid: main

Área Grid: sidebar (Ancho: 50px)

Al hacer clic en Categoría

Se superpone

#sidebar

Ancho: 50px

Z-index: 10

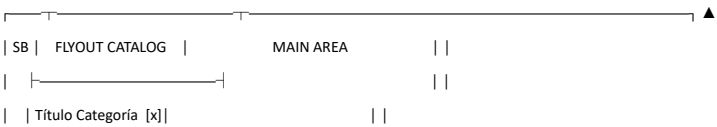
.catalog-flyout

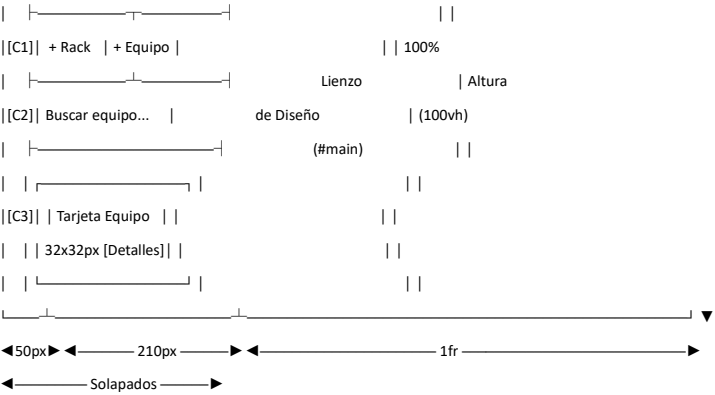
Ancho: 210px

Z-index: 50

Lienzo Central (#main)

 Distribución de Píxeles (Sidebar y Catálogo Expandido)





Dimensiones y Medidas del Panel Inferior (Bottom Panel Layout)

Este documento detalla el esquema de layout, dimensiones de los componentes, celdas, tipografías y comportamiento responsivo de la sección de tablas inferiores (#bottom) de **RACK Designer Next**.

1. Estructura y Dimensiones Estructurales

El panel inferior actúa como una hoja de datos tipo Excel interactiva que contiene el Inventario global de hardware o las Conexiones físicas del datacenter.

```
#bottom {
  grid-area: bottom;
  background: var(--bg-card);
```

```
border-top: 1px solid var(--border);

display: flex;

flex-direction: column;

transition: height 0.2s ease;

height: var(--bottom-h); /* 220px */

overflow: hidden;

z-index: 50;

}
```

Estados de Altura Dinámicos

- Expandido (Por Defecto): Altura fija de **220px** (--bottom-h).
- Colapsado (Clase .collapsed): Altura reducida a **38px**, ocultando las tablas de datos y mostrando únicamente la cabecera de pestañas.
- Pantalla Completa en Móvil (Clase .fullscreen): Toma una altura del **85vh** del dispositivo para permitir operar con comodidad las tablas en pantallas reducidas.

2. Cabecera del Panel (.bottom-header)

La cabecera contiene los selectores de pestañas (Inventario y Conexiones), botones de acción rápida, filtros de búsqueda y exportaciones.

- Altura Fija: **38px** (flex-shrink: 0).
- Layout: Flexbox horizontal (display: flex, align-items: center, gap: 8px) con relleno lateral de padding: 0 14px.
- Elementos Normalizados a Altura de **24px**:
 - Pestañas de Control (.tab-pill): Relleno interno de 5px 14px. El botón activo tiene fondo iluminado (var(--accent-glow)) y texto de color de énfasis (var(--accent)).

- **Buscador Local (#table-search):** Ancho fijo de **160px**.
- **Botón de Menú de Exportación (#btn-export-menu):** Despliega el menú #export-dropdown posicionado en z-index: 9999 con un ancho de **180px**.

3. Contenedor de Tablas (.table-wrap)

El contenedor de datos se expande para absorber todo el espacio restante por debajo de la cabecera.

```
.table-wrap {  
  flex: 1;  
  
  overflow: auto; /* Permite scroll vertical y horizontal independiente */  
}
```

4. Estructura de la Tabla de Datos (table.data-table)

Las tablas de datos utilizan el modelo de colapso de bordes y tipografías monoespaciadas para la alineación precisa de metadatos (como IPs y números de serie).

- **Anchura Base:** 100% (border-collapse: collapse).
- **Encabezados (th):**
 - Relleno: padding: 6px 12px.
 - Fuente: Tamaño --text-xs (10px), grosor 60px y tipografía monoespaciada (var(--font-mono) = JetBrains Mono).
 - **Cabecera Sticky:** Cuenta con position: sticky; top: 0; para mantenerse fija al hacer scroll vertical en la lista de equipos.
 - Ajuste: white-space: nowrap para evitar saltos de línea molestos.
- **Celdas (td):**
 - Relleno: padding: 5px 12px.

- Fuente: Tamaño --text-sm (11px), tipografía monoespaciada y color secundario (var(--text-secondary)).
- Ajuste: white-space: nowrap y alineación vertical media.

- **Fila en Hover (tr:hover td):** El fondo cambia a un color de tarjeta destacado (var(--bg-card2)) para mejorar el seguimiento de la fila.

5. Edición Rápida de Celdas (input.cell-edit)

Al hacer doble clic sobre celdas con la clase .editable, se inyecta un campo de edición en línea:

- **Campo de Entrada (.cell-edit):**
 - Dimensiones: width: 100%, relleno de 2px 6px y bordes de 3px.
 - Contorno: Destacado con borde 1px solid var(--accent) y color de texto principal.
- **Animación de Error (.error):**
 - Contorno: Cambia a rojo (var(--red)).
 - Efecto: Sacudida horizontal (shake 0.3s) y sombra de resplandor roja (var(--red-glow)).

6. Micro-Elementos y Badges

- **Insignias de Tipo (.type-badge):**
 - Relleno: 1px 6px con bordes redondeados de 3px.
 - Fuente: Tamaño --text-xs (10px), negrita (700) y tipografía monoespaciada.
 - Colores: Fondos atenuados de baja opacidad al 15% (rgba(..., 0.15)) con texto brillante (ej. Servidores en azul celeste, Switches en verde, Routers en ámbar).
- **Botones de Fila (.tbl-action):**
 - Dimensiones: Relleno de 2px 7px con bordes de 3px.

- Hover: Cambia su borde y texto a rojo (`var(--red)`) para el botón de eliminar.
- **Indicador de Cable (.cable-dot):** Círculo perfecto de **10px × 10px** (`border-radius: 50%`) pintado dinámicamente con el color hexadecimal del cable.